

Beyond simple pipelining

how can we achieve higher performance than possible with just the simple pipelining we've looked at so far?

deep pipelining (also called “super-pipelining”)

break up instruction processing into a **greater number of steps**

current common processors typically have pipelines with **10-20** stages for integer operations

superscalar (also called “dynamic multiple issue”)

with this approach, **start** (“issue”) **multiple instructions per clock cycle**

up to hardware to check for data hazards

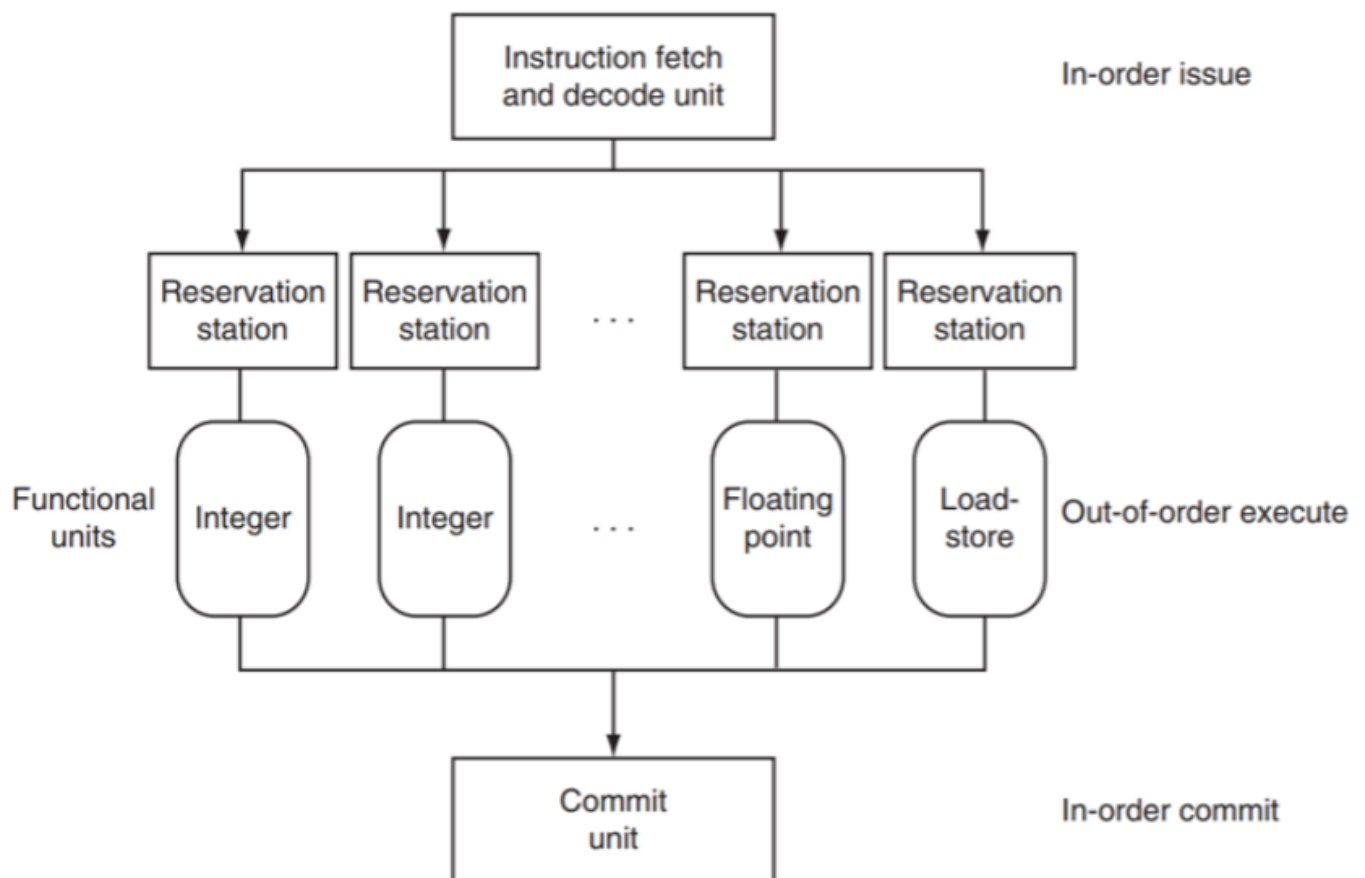
not all components of pipeline replicated to the same degree, so must also deal with the possibility of **structural hazards**

out-of-order execution (“dynamic pipeline scheduling”)

allow instructions to **execute out of order**, so that an instruction that takes longer to process does not hold up all the instructions behind it

although can *execute* out of order, do require **in-order commit**

high-level picture (from text Fig. 4.73)



speculation

superscalar, out-of-order execution processors make heavy use of *speculation*, including both **branch prediction**, and **load address speculation**

example code for load address speculation:

```
sw t1,100(s0)
```

```
lw t2,4(s2)
```

can the "**lw**" be executed before the "**sw**"?

VLIW ("very long instruction word", also called "static multiple issue")

like superscalar / dynamic multiple issue approach, multiple instructions issued per clock cycle

but with this approach, it's the **compiler's responsibility** to group together **independent instructions** to be issued together

motivation is simpler hardware, but approach is less popular than dynamic multiple issue approach

security implications of speculation and out-of-order execution

security attacks made possible by speculation and out-of-order execution have been discovered (**Meltdown**, **Spectre**)

speculation and out-of-order execution do not change the *functional results* of code execution

- for example, if an attempt to read protected info should generate an exception, it will still generate an exception if using speculation and out-of-order execution

speculation and out-of-order execution may, however, change the **microarchitecture state** (e.g. contents of caches) resulting from code execution

can these changes to microarchitecture state be made to **depend on protected info**?

- if so, then may be possible to use *timing attacks* to reveal this info

a number of mitigations have been implemented, e.g. [KPTI](#), but completely addressing these vulnerabilities may require substantial system design changes

performance limitations

deep pipelining, superscalar, out-of-order execution, speculation, VLIW ... all these techniques exploit ILP (“instruction level parallelism”) in ordinary sequential programs

unfortunately, experience has shown that the amounts of ILP we can exploit when executing common software applications are quite limited
further improving performance requires software written to explicitly exploit parallelism

at the hardware level, multiple approaches to supporting parallel execution, including: multicore processors, vector instructions, GPUs, accelerators
software is written to exploit these hardware capabilities, e.g. using multi-threaded programs, compiler support, language extensions, libraries

will take a brief look at parallel computing, time permitting, as our last topic in the course ...